UNIEVANGÉLICA · ENGENHARIA DE SOFTWARE
PROJETO INTEGRADOR · 6º PERÍODO · FASE 2

Pro Service
Integração entre frontend e backend

Responsável por esta entrega: Natanael Cardoso da Silva
Matrícula: 2410521 · Outubro de 2026

22.3 Objetivo e escopo da integração

Esta etapa conecta a interface do Pro Service aos serviços da API, preservando o propósito da plataforma: aproximar clientes e prestadores de serviços. A implementação contempla autenticação, busca de profissionais, manutenção do perfil, associação de habilidades, contratação, acompanhamento de status e avaliações.

O trabalho utiliza como base o frontend React com perfis de cliente e prestador e o backend corrigido da fase anterior, construído em Node.js, Express e MySQL. As telas que apresentavam dados fixos e autenticação apenas local foram substituídas por consultas e operações HTTP. O servidor permanece responsável pelas regras de negócio e pela persistência.

22.3.1 Arquitetura adotada

Camada	Responsabilidade
React e React Router	Capturar dados, apresentar resultados da API e controlar a navegação por sessão.
Cliente HTTP centralizado	Enviar JSON e token Bearer; interpretar erros e limitar o tempo de espera.
Express: rotas, controllers e services	Validar requisições, autenticar, autorizar e aplicar as regras do sistema.
Repositories e MySQL	Executar consultas parametrizadas e manter os registros relacionais.

Situação de validação: o código foi integrado e o frontend compilou. Os testes HTTP descritos neste relatório passaram. A execução ponta a ponta com MySQL continua pendente porque o ambiente de preparação não possui um servidor disponível na porta 3306. Este relatório não apresenta essa etapa como concluída.

22.3.2 Comunicação entre as camadas

A interface utiliza a função central api(), localizada em frontend/src/services/api.js. Cada operação encaminha uma requisição à API versionada em /api/v1. Respostas JSON alimentam as telas; falhas de rede e erros de validação são exibidos ao usuário. Os formulários de gravação bloqueiam novas submissões durante o processamento.

Funcionalidade	Contrato HTTP principal
Cadastro e login	POST /auth/cadastro; POST /auth/login
Recuperação da sessão	GET /auth/me
Busca e perfil público	GET /prestadores; GET /prestadores/:id
Perfil do prestador	GET e PATCH /prestadores/me
Habilidades	GET /habilidades; POST /prestadores/me/habilidades; DELETE /prestadores/me/habilidades/:id
Solicitação e acompanhamento	POST /servicos; GET /servicos/meus; GET /servicos/oportunidades
Status do atendimento	PATCH /servicos/:id/status
Avaliação	POST /avaliacoes; GET /avaliacoes/prestador/:id

Todos os caminhos acima recebem o prefixo /api/v1. A busca e o acompanhamento utilizam paginação retornada pelo servidor. O identificador usado para consultar ou contratar um prestador é usuarioId, coerente com o contrato do backend; ele não é confundido com o identificador interno do perfil.

22.3.3 Autenticação e tratamento da sessão

O cadastro e o login dependem da resposta do servidor. O perfil CLIENTE ou PRESTADOR é obtido dos dados autenticados, e não escolhido livremente na tela de login. O token fica em sessionStorage; ao recarregar a interface, a aplicação consulta /auth/me antes de liberar as telas protegidas.

Requisições protegidas enviam Authorization: Bearer. Respostas 401 invalidam a sessão quando aplicável. Senhas são tratadas com bcrypt no backend. A proteção das rotas da interface complementa a autorização do servidor, que verifica as permissões independentemente da navegação exibida.

22.3.4 Fluxo funcional integrado

1. Preparação do prestador. Após cadastrar sua conta, o profissional informa descrição, cidade, UF, preço por hora e experiência. A interface grava o perfil e permite adicionar ou remover habilidades. O backend exige ao menos uma habilidade para o profissional aparecer na busca e receber solicitações.

2. Solicitação do cliente. O cliente consulta os profissionais, acessa o perfil e envia uma solicitação com descrição do trabalho e local de atendimento. O backend valida os campos, confere o prestador e grava o serviço com status PENDENTE. A interface então abre a lista de serviços obtida da API.

3. Execução do atendimento. O prestador responsável pode aceitar e concluir o serviço. Cliente e prestador podem cancelar enquanto o estado permitir. O fluxo admite PENDENTE para ACEITO ou CANCELADO; ACEITO para CONCLUIDO ou CANCELADO. Estados finais não admitem novas transições.

4. Avaliação. O cliente participante pode avaliar um serviço concluído com nota de 1 a 5. A API bloqueia avaliações anteriores à conclusão, de terceiros ou duplicadas. A lista de serviços recebe o identificador da avaliação e apresenta a nota já registrada. O perfil público consulta a média e os comentários persistidos.

22.3.5 Consistência e persistência

O cadastro do prestador mantém a transação do backend para criar usuário e perfil de forma atômica. Consultas e gravações permanecem na camada de repositórios. Na atualização de status, o UPDATE exige também o estado anterior esperado, impedindo que uma alteração concorrente seja sobrescrita sem aviso; nesse caso, a API orienta atualizar a lista.

A inicialização do banco foi ajustada para usar CREATE TABLE IF NOT EXISTS, sem apagar dados existentes. O comando npm run db:setup lê as variáveis do arquivo .env e prepara a estrutura e as habilidades iniciais. Esse procedimento não converte automaticamente bancos com estruturas antigas incompatíveis.

O frontend não usa listas de negócio simuladas nem grava solicitações no armazenamento do navegador. A atualização entre contas ocorre ao consultar ou atualizar a lista; não há mecanismo de tempo real nesta versão. A rota /health verifica a conexão com o banco e responde 503 quando ele está indisponível.

22.3.6 Configuração de execução

O arquivo LEIA_PRIMEIRO.md reúne os comandos completos. O backend usa a porta 3000; o frontend usa a porta 5173. Durante o desenvolvimento, o proxy do Vite encaminha /api ao Express. Em hospedagens separadas, VITE_API_URL deve apontar para a API real e FRONTEND_URL deve permitir a origem do frontend. Segredos e credenciais ficam no .env local.

22.3.7 Verificação e evidências disponíveis

Verificação	Resultado observado
Sintaxe do backend	Arquivos JavaScript de src, scripts e tests verificados com node --check, sem erro de sintaxe.
Build do frontend	Vite 5.4.21: 44 módulos transformados e build de produção concluído. Arquivos gerados em frontend/dist.
HTTP e CORS	Oito cenários aprovados; nove testes no total contando o agrupador do Node.js.
Integração com MySQL	Tentativa bloqueada por ECONNREFUSED 127.0.0.1:3306, antes das operações do fluxo. Não aprovada nem demonstrada neste ambiente.
Teste visual em navegador	Não executado: o navegador automatizado não estava disponível. O build não substitui essa verificação.

Os cenários HTTP verificaram rota protegida sem token, token inválido, cadastro incompleto, login com dados inválidos, identificador inválido, paginação inválida, rota inexistente e preflight CORS de origem autorizada. Esses testes exercitam a aplicação Express, mas não comprovam gravação no banco.

O arquivo backend/tests/integration.test.js foi preparado para usar MySQL real, sem substituir os repositórios por simulações. Ele verifica cadastro, login, perfil, habilidade, busca, criação de serviço, consulta SQL, listagem nas duas contas, autorização, aceite, conclusão, avaliação, duplicidade e cancelamento. Deve ser executado no ambiente configurado antes da apresentação.

22.3.8 Demonstração exigida para a avaliação

O roteiro docs/DEMONSTRACAO.md orienta a execução em duas sessões. A evidência central consiste em mostrar o mesmo identificador do serviço na tela, na resposta HTTP e no registro MySQL. Em seguida, o prestador altera o status e o cliente consulta a atualização e registra a avaliação.

Também deve ser demonstrada a persistência após reiniciar a API, além da rejeição de entradas inválidas e operações sem permissão. As capturas dessas etapas precisam ser realizadas no ambiente real; nenhuma imagem de gravação em banco foi fabricada para esta entrega.

22.3.9 Resultado e condição de encerramento

A implementação entrega uma interface conectada aos contratos do backend e conserva a arquitetura adotada pelo projeto. A fase estará tecnicamente demonstrada quando o teste de integração com MySQL for aprovado e o percurso completo for verificado no navegador. Até esse momento, o resultado é código integrado com validação parcial, e não comprovação definitiva de operação ponta a ponta.